→ Additionally, $f := k+r = \lceil \frac{k}{\ell} \rceil \cdot \ell < (\frac{k}{\ell}+1) \cdot \ell = k+\ell \leq n+1$.
↳ D_n has $k+\ell$ vertices

→ **Claim:** Let $t \leq n$ with $x_t = x_{2t}$, then: $x_{t+k} = x_k$ ⇒ **Proof:** $x_t = x_{2t} \Rightarrow 2t = t + p \cdot \ell$ for $p \in \mathbb{N} \Rightarrow x_{k+t} = x_k$

Hare & Tortoise Algorithm:

hare → a[a[n]] (2 steps in graph), tortoise → a[n] (1 step in graph), t → 1

```
while (hare ≠ tortoise)
    tortoise → a[tortoise]
    hare → a[a[hare]]
    t → t+1
```
→ cycle-finding portion

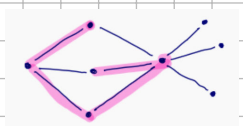
↳ 3 variables in memory

```
hare → n
while (hare ≠ tortoise)
    i → hare
    j → tortoise
    tortoise → a[tortoise]
    hare → a[hare]
```
(2 additional vars. in mem.) → locate i and j with a[i] = a[j]

Long Path Problem:

→ Given a graph $G = (V, E)$ and $\beta \in \mathbb{N}_0$, determine if there exists a path of length β in G.

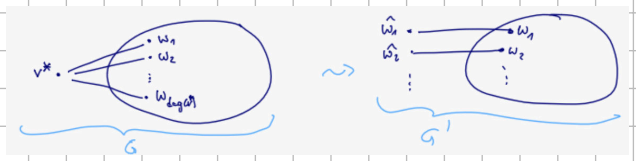
Example:



→ yes for $\beta \leq 4$, remember on a path we cannot repeat vertices

Theorem: If we can decide long-path for graphs with n vertices in time $t(n)$, we can decide if a graph with n vertices has a Hamiltonian cycle in time $t(2n-2) + O(n^2)$. (recall that finding a Hamiltonian cycle is NP-complete)

Proof (optional): We have to show that if $G(V,E)$ has Hamiltonian cycle $\Leftrightarrow$ G' has a path of length n, with G' being a graph we can construct in time $O(n^2)$ with $n' \leq 2n-2$ vertices

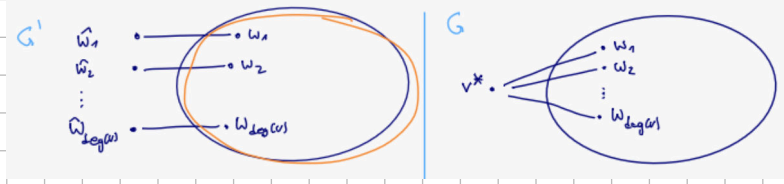


→ Let v^* be any vertex. Remove v^*. Replace the edges $\{v^*, w_1\}, \dots, \{v^*, w_{\deg(v^*)}\}$ with new edges $\{w_1, \hat{w}_1\}, \dots, \{w_{\deg(v^*)}, \hat{w}_{\deg(v^*)}\}$ with $\hat{w}_1, \dots, \hat{w}_{\deg(v^*)}$ being new edges.

→ Show that G has Hamiltonian cycle ⇒ G' has a path of length n:

→ Let $\langle v_1, \dots, v_n, v_1 \rangle$ be a Hamiltonian cycle in G.

→ Assume that $v_1 = v^*$. Then $\langle \hat{v}_2, \dots, v_n, \hat{v}_n \rangle$ is a path of length n in G'.



Now show the opposite direction:

→ Let $\langle u_0, \dots, u_n \rangle$ be a path of length n in G'.

→ Vertices $u_1, \dots, u_{n-1}$ have degree ≥ 2 and are pairwise distinct

⇒ These are the vertices in $V \setminus \{v^*\}$. Therefore, let $u_0 = \hat{w}_i$ and $u_n = \hat{w}_j$ be pairwise distinct new vertices with degree 1 in G'.

⇒ $\langle v^*, u_1, \dots, u_{n-1}, v^* \rangle$ is a Hamiltonian cycle in G.

Goal → solve the long path problem in $O(c^\beta)$ for a constant c, brute force takes $O(n^{\beta+2})$

Color coding:

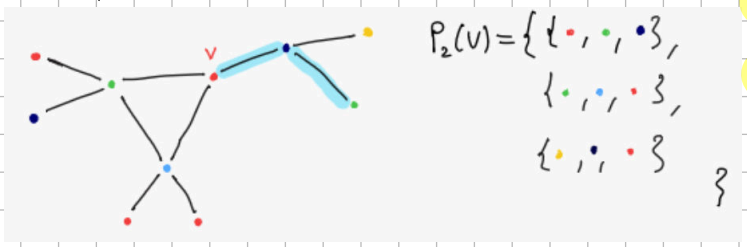
Idea: Color vertices randomly, find "colorful paths" (bunte Pfade)

→ Let $G = (V, E)$ be a graph with a coloring $\gamma : V \to [k]$. A path is colorful if all vertices have different colors $\gamma(v)$.

Checking for colorful paths: Given a graph $G = (V, E)$ and a coloring $\gamma : V \to [k]$, decide if there exists a colorful path of length $k-1$.

→ $\forall v \in V$ and $i \in \{0,\dots,k-1\}$ let $P_i(v) := \{S \subseteq [k] : |S| = i+1$ and $\exists$ a colorful path colored in S that ends with $v\}$

Example:



Observation: $\exists$ colorful path of length $k-1 \Leftrightarrow \bigcup_{v \in V} P_{k-1}(v) \neq \emptyset$

Recursive formula for $P_i(v)$: $P_0(v) = \{\{\gamma(v)\}\}$

For $i \geq 1$: $\exists$ a colorful path colored with S that ends in v

$\Leftrightarrow \exists$ neighbor x of v and a colorful path colored with $S \setminus \{\gamma(v)\}$ that ends in x

→ Therefore, $P_i(v) = \bigcup_{x \in N(v)} \{R \cup \{\gamma(v)\} \mid R \in P_{i-1}(x) \text{ und } \gamma(v) \notin R\}$

Algorithm Colorful(G,i), which computes $P_i(v)$ $\forall v \in V$ given $P_{i-1}(v)$ $\forall v \in V$:

```
1: for all v ∈ V do
2:     P_i(v) ← ∅
3:     for all x ∈ N(v) do
4:         for all R ∈ P_{i-1}(x) mit γ(v) ∉ R do
5:             P_i(v) ← P_i(v) ∪ {R ∪ {γ(v)}}
```

Runtime analysis:

→ # of subsets of $[k]$ with i elements: $\binom{k}{i} \Rightarrow |P_{i-1}(x)| \leq \binom{k}{i}$

→ Runtime of colorful(G,i): $O(\sum_{v \in V} \deg(v) \binom{k}{i} i) = O(\binom{k}{i} \cdot i \cdot m)$ ($m = |E| = \frac{1}{2}\sum_{v \in V} \deg(v)$; i ← size of sets in $P_{i-1}(v)$)

→ Total runtime (running alg. from 1 until k): $\sum_{i=1}^{k-1} \binom{k}{i} i \cdot m \leq \sum_{i=0}^{k} \binom{k}{i} k \cdot m = 2^k \cdot k \cdot m \Rightarrow$ Runtime $= O(2^k \cdot k \cdot m)$

Random colorings:

→ Assume G has a path P of length $k-1$. $\Rightarrow \exists k^k$ colorings of P with k colors, and $k!$ colorful colorings of P with k colors

$\Rightarrow$ If we randomly color the vertices of G, $Pr[\exists$ colorful path$^{(P)}$ of length $k-1] \geq Pr[P$ is colorful$] \geq \frac{k!}{k^k} \geq e^{-k}$

($\frac{k!}{k^k} \geq e^{-k}$ ← Power series e^x: $e^k = \sum_{i=0}^{\infty} \frac{k^i}{i!} \geq \frac{k^k}{k!}$)

Theorem: Let G be a graph with a path of length $k-1$.

1) A random coloring with k colors generates a colorful path of length $k-1$ with probability $p \geq e^{-k}$

2) For repeated colorings with k colors, the expected value of attempts until there is a colorful path of length $k-1$ is $\frac{1}{p} \leq e^k$
   ← in $Geo(p) \Rightarrow E = \frac{1}{p}$

Monte Carlo algorithm for long paths:

① Repeat the following $\lceil \lambda e^k \rceil$ times:

→ color the vertices of G randomly and uniformly with $k = (B+1)$ colors and verify if $\exists$ a colorful path of length $k-1$

→ if yes, return YES

② If no iteration returned YES, return NO

→ Runtime: $O(\lambda \cdot e^k \cdot 2^k \cdot k \cdot m) = O(\lambda \cdot (2e)^k \cdot k \cdot m)$

Theorem: 1) The algorithm has runtime $O(\lambda (2e)^k km)$

2) If the algorithm returns YES, the graph has a path of length $k-1$

3) If the graph has a path of length $k-1$, the probability of the algorithm returning NO is maximally $e^{-\lambda}$

Proof of 3): $Pr[NO] \leq (1-e^{-k})^{\lceil \lambda e^k \rceil} \leq (e^{e^{-k}})^{\lceil \lambda e^k \rceil} = e^{-e^{-k} \cdot \lceil \lambda e^k \rceil} \leq e^{-\lambda}$ ($1+x \leq e^x \ \forall x \in \mathbb{R}$, here $x = e^{-k}$) ∎

→ the algorithm only runs in polynomial time for paths of length $\leq \log(n)$

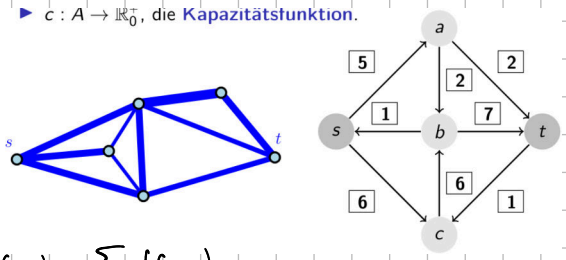
→ deterministic version: **Theorem:** $\exists$ a family F of colorings $\gamma: V \to [2k]$ s.t.:

→ $\forall A \subseteq V$ with $|A| \leq k$: there is a coloring in F s.t. A is colorful with coloring γ

→ $|F|$ has polynomial size, meaning it can be constructed in polynomial time
$k = \mathcal{O}(\log n)$

Flow in networks:

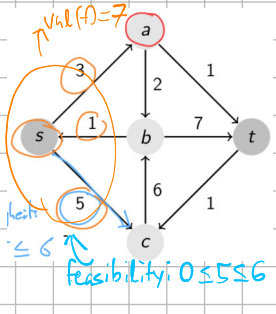
Def.: A network is a tuple $N = (V, A, c, s, t)$ with (V, A) being a directed acyclic graph (DAG), $s \in V$ being the source, $t \in V \setminus \{s\}$ being the sink, and $c: A \to \mathbb{R}_0^+$ being the capacity function



Def.: Let $N = (V, A, c, s, t)$ be a network. A flow in N is a function $f: A \to \mathbb{R}$ with the following conditions:

Feasibility: $0 \leq f(e) \leq c(e)$ $\forall e \in A$, **flow conservation:** $\forall v \in V \setminus \{s,t\}: \sum_{u \in V: (u,v) \in A} f(u,v) = \sum_{u \in V: (v,u) \in A} f(v,u)$

Value of a flow: $\text{val}(f) = \text{netoutflow}(s) := \underbrace{\sum_{v \in V: (s,v) \in A} f(s,v)}_{\text{outflow}} - \underbrace{\sum_{v \in V: (v,s) \in A} f(v,s)}_{\text{inflow}}$



Lemma: The net inflow of the sink t equals the value of the flow:

$\text{netinflow}(t) := \sum_{v \in V: (v,t) \in A} f(v,t) - \sum_{v \in V: (t,v) \in A} f(t,v) = \text{val}(f)$

Proof: $0 = \sum_{(v,v) \in A} f(v,v) - \sum_{(v,v) \in A} f(v,v) = \sum_{v \in V} \left(\underbrace{\sum_{u \in V: (v,u) \in A} f(v,u) - \sum_{u \in V: (u,v) \in A} f(u,v)}_{0 \text{ for } v \neq \{s,t\} \text{ (flow conservation)}} \right) = \underbrace{\left(\sum_{u \in V: (s,u) \in A} f(s,u) - \sum_{u \in V: (u,s) \in A} f(u,s) \right)}_{= \text{val}(f)}$

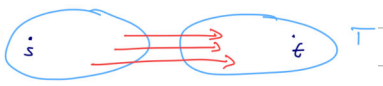
$+ \underbrace{\left(\sum_{v \in V: (t,v) \in A} f(t,v) - \sum_{v \in V: (v,t) \in A} f(v,t) \right)}_{= -\text{netinflow}(t)}$ ∎

Maxflow Problem: Given a network $N = (V, A, c, s, t)$, find a flow f with the highest value (a max. flow)

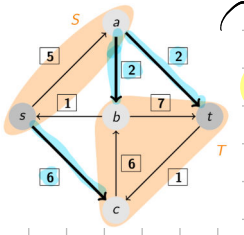
→ Does this always exist? How would you know if a flow is maximal?

Def.: An s-t cut for a network (V, A, c, s, t) is a partition (S, T) of V with $s \in S$ and $t \in T$. ($S \cup T = V$, $S \cap T = \emptyset$)

The capacity of an s-t cut (S, T) is $\text{cap}(S, T) := \sum_{(v,w) \in (S \times T) \cap A} c(v,w)$ ← edges from S to T, not T to S



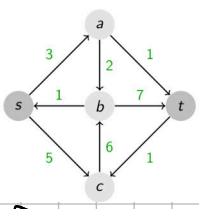
↷ $\text{cap}(S, T) = 6 + 2 + 2 = 10$



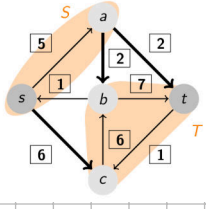
Lemma: Let f be a flow and (S, T) an s-t cut. $\text{val}(f) \leq \text{cap}(S, T)$ (I will omit the proof)

Consequence: If for a flow f, we find an s-t cut (S, T) with $\text{cap}(S, T) = \text{val}(f)$, f is a maximal flow.

⇒ (S, T) is a simple certificate for the maximality of f.



Flow with $\text{val}(f) = 7$



s-t cut with $\text{cap}(S, T) = 10$

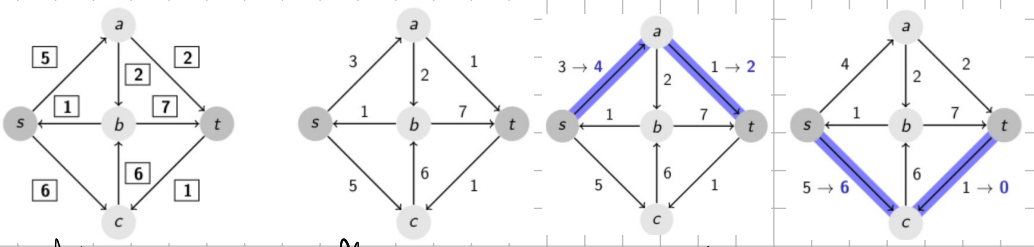
Maxflow-Mincut Theorem: For any network $N = (V, A, c, s, t)$,

$\max_{\text{flow } f} \text{val}(f) = \min_{s\text{-}t \text{ cut } (S,T)} \text{cap}(S, T)$

⇒ s-t-mincut is a finite algorithmic problem and a min. cut always exists (there is a finite amount of s-t cuts)

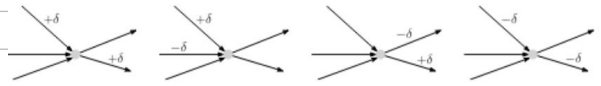
Goal: Algorithm and proof for the aforementioned theorem

Improving a flow:



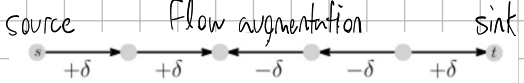
Network N | flow f, val(f) = 7 | val(f) = 8 | val(f) = 9

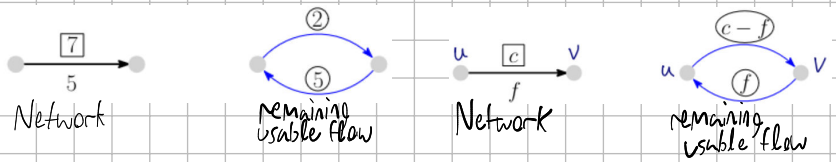
We can change the flow while maintaining flow conservation in the following ways:



→ considering: { the capacity for "+δ"
the current flow for "−δ"

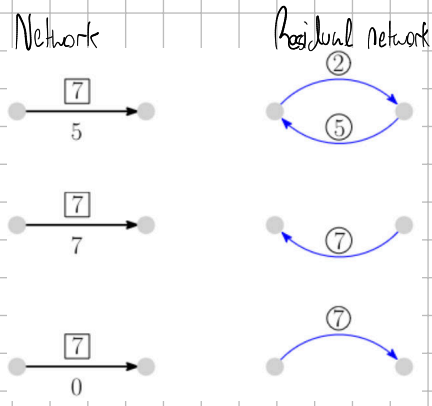
Finding augmenting paths / handling remaining flow:





→ Notation: For $e = (v,w)$, let $e^{opp} := (w,v)$

Def.: Let $N = (V, A, c, s, t)$ be a network (without opposite edges) and f be a flow in N. The residual network $N_f := (V, A_f, r_f, s, t)$ is defined as:



1) If $e \in A$ with $f(e) < c(e)$, e is an edge in A_f with $r_f(e) := c(e) - f(e)$

2) If $e \in A$ with $f(e) > 0$, e^{opp} is an edge in A_f with $r_f(e^{opp}) = f(e)$

3) A_f only contains edges as outlined in 1) and 2)

→ The value $r_f(e)$ is known as the remaining capacity of an edge e

Theorem: Let N be a network (without opposite edges).

A flow f is maximal if and only if the residual network N_f has no directed s-t path.

Each maximal flow f has an s-t cut $(S, \overline{T})$ with $val(f) = cap(S, \overline{T})$

Proof: Claim: The residual network N_f has a directed s-t path $\Rightarrow$ f can be augmented ($\Leftrightarrow$ f not maximal)

Proof: Consider a directed s-t path in N_f: 

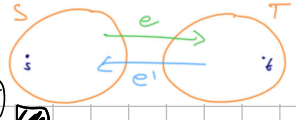
and take the smallest residual capacity $\varepsilon = \min_i \varepsilon_i$, then augment f along the path around ε

Claim: The residual network N_f has no directed s-t path $\Rightarrow$ $\exists$ s-t cut $(S, \overline{T})$ with $cap(S, \overline{T}) = val(f)$ ($\Rightarrow$ f maximal)

Proof: $S :=$ vertices reachable from S in N_f, $\overline{T} := V \setminus S$ $\Rightarrow$ $s \in S$, $t \notin S$ (no directed s-t path in N_f) $\Rightarrow$ $(S, \overline{T})$ is s-t cut

Claim: $cap(S, \overline{T}) = val(f)$ (using S and $\overline{T}$ we defined in last proof, recall $(S \times \overline{T}) \cap A_f = \emptyset$)

Proof: $\forall e \in (S \times \overline{T}) \cap A: f(e) = c(e) \Rightarrow f(S, \overline{T}) = cap(S, \overline{T})$
$\forall e' \in (\overline{T} \times S) \cap A: f(e') = 0 \Rightarrow f(\overline{T}, S) = 0$
$\Rightarrow val(f) = \overbrace{f(S, \overline{T})}^{= cap(S,\overline{T})} - \overbrace{f(\overline{T}, S)}^{=0} = cap(S, \overline{T})$ ∎



Ford-Fulkerson Algorithm:

1: $f \to 0$ (flow constant 0) → we cannot guarantee that the algorithm terminates

2: while $\exists$ s-t path P in N_f do (augmenting path) → the algorithm may run infinitely for capacities in $\mathbb{R}$

3: Augment flow along P → for capacities in $\mathbb{N}_0$, the flows and remaining capacities stay integers, and the flow value is increased by 1 each augmenting step

4: return f (max. flow)

Runtime analysis: Let $n := |V|$ and $m := |A|$ for a network $N = (V, A, c, s, t)$

→ Assume $c: A \to \mathbb{N}_0$ and $U := \max_{e \in A} c(e)$. Then, $val(f) \leq cap(\{s\}, V \setminus \{s\}) \leq (n-1)U$, meaning there are at most $(n-1)U$ augmenting steps

→ One augmenting step (search for s-t path in N_f, augment, update N_f) is done in $O(m)$ $\Rightarrow$ total runtime $O(m \cdot n \cdot U)$

Theorem (Ford-Fulkerson with integer capacities): Let $N = (V, A, c, s, t)$ be a network with $c: A \to \mathbb{N}_0^{\leq U}$ (for $U \in \mathbb{N}$) (without opposite edges). Then, there is an integer maximal flow that can be computed in $O(mnU)$

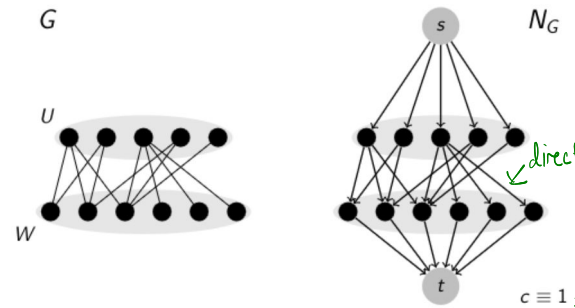
→ That concludes our proof of the Maxflow-Mincut Theorem, I think this week's peer graded exercise (P5) is great practice

Practical uses of flow:

Max. Bipartite Matching Problem: Find a maximal matching for an unweighted, undirected graph:

→ Matchings are covered on page 5 if you want a recap

→ The greedy algorithm won't be maximal → we solve this by reducing it to a maxflow problem



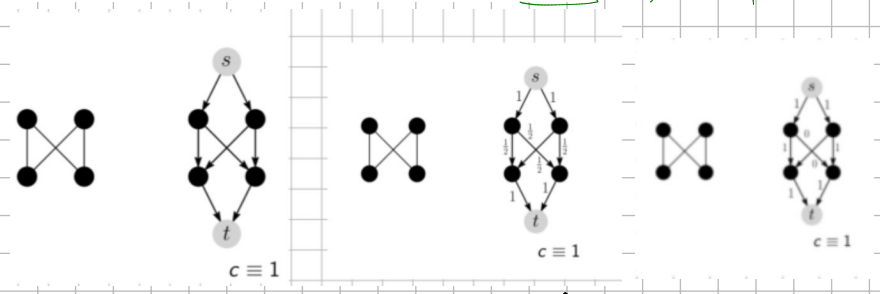
→ We map each bipartite graph with given vertex partition $U \dot\cup W$ onto a network:

$G = (U \dot\cup W, E) \mapsto N_G = (U \dot\cup W \dot\cup \{s,t\}, A, c, s, t)$ (bipartite graph; network; edges)

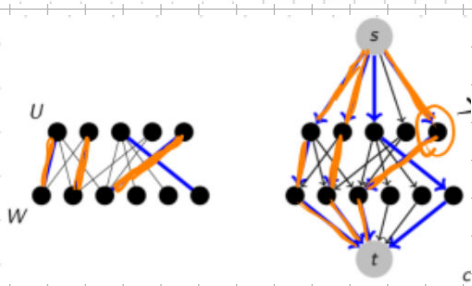
→ $s \neq t$ are added vertices

→ $A := \{s\} \times U \cup \bigcup_{(u,w) \in U \times W \mid \{u,w\} \in E} \{(u,w)\} \cup W \times \{t\}$

→ $c \equiv 1$.



→ By the Theorem for Ford-Fulkerson, there is an integer maxflow computable in $O(mnU)$ with $U \equiv 1$ here



→ A matching M in G maps to an integer flow f_M in N_G with $val(f_M) = |M|$

→ either 0 flows in and 0 flows out or 1 flows in and out of a vertex → Matching condition: we cannot choose 2 neighboring vertices $\Rightarrow$ the integer flow f in N_G also maps to the matching

$\Rightarrow$ max. matching in G "$\cong$" integer maxflow in N_G

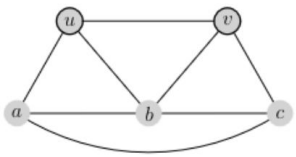
→ $\max_{M \text{ matching in } G} |M| = \max_{f \text{ integer flow in } N_G} val(f) = \max_{f \text{ flow in } N_G} val(f)$

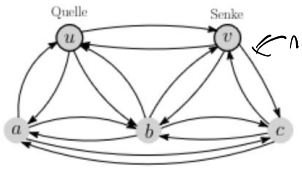
Edge-disjoint paths:

Problem: Given G with 2 selected edges u, v, $u \neq v$, compute the max. set of edge-disjoint u-v paths

→ Recap on Menger on page 1

→ We map $\hat{G}=(V,E), u,v\in V \mapsto N_G^{uv}=(V,A,c,u,v)$, $A:=\{(x,y),(y,x) \mid \{x,y\}\in E\}$, $c\equiv 1$ (graph with 2 vertices → network)



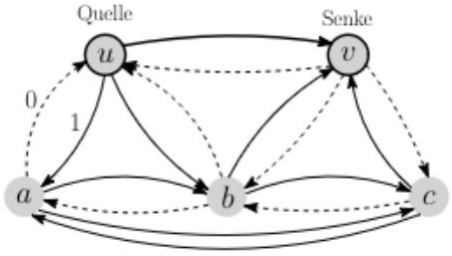


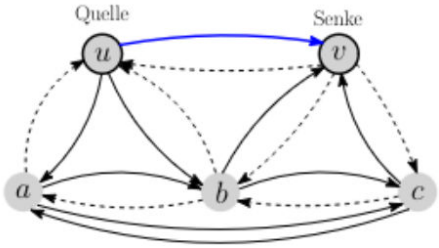
← notice how opposite edges are allowed

→ Compute an integer maxflow f in N_G^{uv} ⇒ flow values $\in\{0,1\}$

→ $\forall w\notin\{u,v\}: indeg_f(w)=outdeg_f(w)$

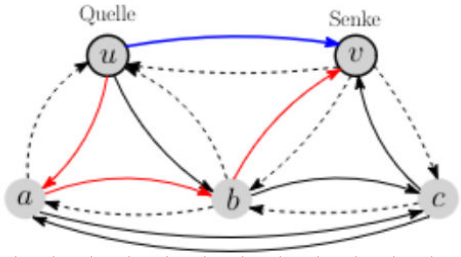
→ $val(f)=outdeg_f(u)-indeg_f(u)=indeg_f(v)-outdeg_f(v)$ (in/out degrees concerning edges with flow = 1)

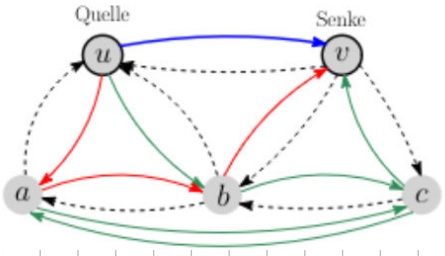


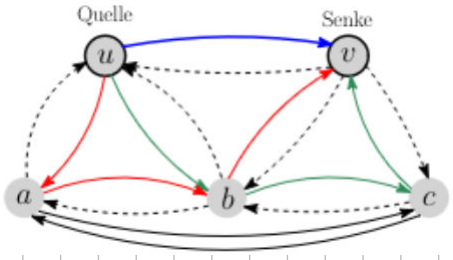


→ starting at u, go along unused edges with flow = 1 until v is reached. any edge used along the way is marked as such.

→ repeat $val(f)$ times. we get $val(f)$ edge-disj. paths







We just proved **Menger's Theorem** in a roundabout way: Let G be a graph with vertices u and v, $u\neq v$:

Max # edge-disjoint paths in G = maxflow(N_G) = min cap(S,T) for u-v-cut (S,T) in N_G = min #edges separating u and v

Mincut Problem:

Def.: A **multigraph** is an undirected, unweighted graph $G=(V,E)$ without loops, but allowing for multiple edges between a single pair of vertices (we can also see this as integer weights)

Def.: An **edge cut** in a multigraph $G=(V,E)$ is a set of vertices C so that $(V,E\setminus C)$ is not connected.

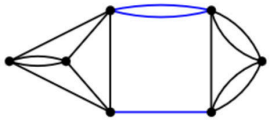
Def.: The **cardinality of a minimum cut** in G, $\mu(G)$, is defined as $\mu(G):=\min_{C\subseteq E,(V,E\setminus C)\text{ not connected}}|C|$

(Equivalently, we can see a cut as a partition $V=S\uplus T$ with $S\neq\emptyset, T\neq\emptyset$, and minimize the edges between S and T)

Mincut problem: Given a multigraph G, compute $\mu(G)$.

→ for G that is not connected, $\mu(G)=0$

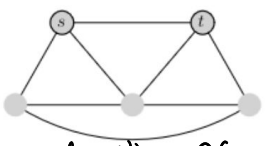
$\mu=4$
min deg $=4$

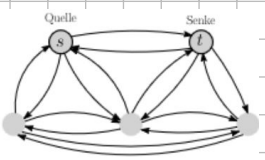


$\mu=3$, min deg $=4$

Def.: For a multigraph $G=(V,E)$, the **degree** $deg(v)=deg_G(v)$ of a vertex is the amount of **incident edges** (not neighbors), therefore: $|E|=\frac{1}{2}\sum_{v\in V}deg(v)$ and $\mu(G)\leq\min_{v\in V}deg(v)$.

Naive approach:





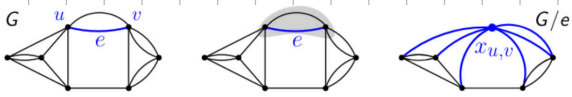
→ we can already compute the smallest s-t cut → smallest amount of edges to remove to separate s from t → this is done in $O(mnU)=O(mn(1+\log U))$ or $O(mn\log n)$

→ Now, fix one s and consider all $t\in V\setminus\{s\}$ → every cut is an s-t cut for some $t\in V\setminus\{s\}$

→ repeat $O(mn\log n)$ $(n-1)$ times ⇒ $O(mn^2\log n)=O(n^4\log n)$

Observation: the lecture also used an em-dash

Next approach ≙ edge contraction: Given: $G=(V,E)$, $e=\{u,v\}\in E$:



Contraction of e: melts u,v to a new edge $x_{u,v}$ that is incident to all edges that were incident to u or v, edges between u and v disappear.

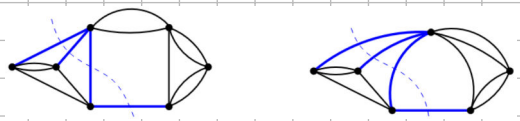
→ We call the new graph G/e. For $k:=$ #edges between u and v: $\deg_{G/e}(x_{u,v}) = \deg_G(u)+\deg_G(v)-2k$ and $|E(G/e)| = |E(G)|-k$

→ Let $e=\{u,v\}$. ∃ a natural bijection:

$E(G)$ without edges between u and v → $E(G/e)$.

→ For $w,w' \in V(G)\setminus\{u,v\}$: $\{w,w'\}\mapsto\{w,w'\}$, $\{w,u\}\mapsto\{w,x_{u,v}\}$, $\{w,v\}\mapsto\{w,x_{u,v}\}$

→ This brings about a bijection: Cuts in G without e → all cuts in G/e



Lemma: Let $G=(V,E)$ be a multigraph and $e\in E$. Then, $\mu(G/e)\geq\mu(G)$.

If G has a mincut C with $e\notin C$, $\mu(G/e)=\mu(G)$.

→ μ can never fall by contracting some edge e, μ stays the same if there exists a mincut without e

How do we find an edge e that contains μ?

Algorithm:

CUT(G) — G connected multigraph

```
G' ← G
while |V(G')| > 2 do
    e ← uniformly distributed random edge in G'
    G' ← G'/e
return size of unique cut in G'
```

→ Let $n:=|V(G)|$. Prerequisites: Contraction and random selection in $O(n)$

→ (this requires viewing multiple edges as edge weights) → CUT(G) runs in $O(n^2)$

→ how does the computed value help us?

→ The algorithm never returns a value $<\mu(G)$

→ If there is a minimal cut C from which no edge is contracted, the algorithm returns $\mu(G)$.

Lemma: Let $G=(V,E)$, $n:=|V|$. for an edge e that is randomly selected from the edges in G, $Pr[\mu(G)=\mu(G/e)]\geq 1-\frac{2}{n}$.

Def.: Success probability $\hat{p}(G)$ of CUT(G) returning $\mu(G)$: $\hat{p}(n):=\inf_{G=(V,E),|V|=n}\hat{p}(G)$

Lemma: $\forall n\geq 3$: $\hat{p}(n)\geq(1-\frac{2}{n})\cdot\hat{p}(n-1)$

Lemma: $\forall n\geq 2$: $\hat{p}(n)\geq\frac{2}{n(n-1)}=1/\binom{n}{2}$ ⇒ expected value of repetitions until we first return $\mu(G)$ is at most $\binom{n}{2}$.

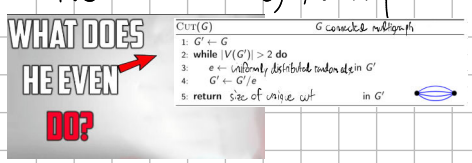
Idea: We repeat CUT(G) $\lambda\binom{n}{2}$ times for some $\lambda>0$ and return the smallest value ever gotten

Theorem: For the algorithm consisting of $\lambda\binom{n}{2}$-time repetition of CUT(G):

1) The algorithm has a runtime of $O(\lambda n^4)$. 2) The smallest encountered value is equal to $\mu(G)$ with a probability of at least $1-e^{-\lambda}$.

→ With $\lambda:=\ln(n)$, we have time $O(n^4\log n)$ with probability of error $\leq 1/n$.

→ one has to ask:



and actually, we can improve it using bootstrapping:

$$\hat{p}(n) \geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \underbrace{\frac{3}{5}}_{0.6} \cdot \underbrace{\frac{2}{4}}_{0.5} \cdot \underbrace{\frac{1}{3}}_{0.33} \cdot \underbrace{\hat{p}(2)}_{=1} = \frac{2}{n(n-1)}$$

CUT(G) — G connected multigraph

1: $G' \leftarrow G$
2: **while** $|V(G')| > 2$ **do**
3: $e \leftarrow$ uniformly distributed random edge in G'
4: $G' \leftarrow G'/e$
5: **return** result of $O(t^4)$ algorithm on G'

→ we can increase the probability of success by being more careful with the last steps → Contract at G' with t vertices, then use the randomized $O(t^4)$ algorithm:

→ $\hat{p}_t(G) :=$ Probability, that $\mu(G)$ is returned

→ $\hat{p}_t(n) := \inf_{G=(V,E),|V|=n} \hat{p}_t(G)$, $\hat{p}_t(t) \geq 1 - e^{-1} = \frac{e-1}{e}$

→ $\hat{p}_t(n) \geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \frac{t+1}{t+3} \cdot \frac{t}{t+2} \cdot \frac{t-1}{t+1} \cdot \hat{p}_t(t) \geq \underbrace{\frac{t(t-1)}{n(n-1)} \cdot \frac{e-1}{e}}_{\bar{p}_t(n) :=}$

→ $\lambda \frac{1}{\bar{p}_t(n)}$-time repetition gives error probability $\leq e^{-\lambda}$ and runtime:

$\underbrace{\lambda \frac{n(n-1)}{t(t-1)} \frac{e}{e-1}}_{\#\text{repetitions}} \cdot O(\underbrace{n(n-t)}_{\text{reduction to } t \text{ vertices}} + \underbrace{t^4}_{\text{alg. over } t \text{ edges}}) = O(\lambda(\frac{n^4}{t^2} + n^2 t^2))$, for $t := \sqrt{n}$, success prob. $\geq 1 - e^{-\lambda}$ and runtime $O(\lambda n^3)$

→ If we do this with the $O(n^3)$ algorithm instead of $O(n^4)$, we get a better algorithm and so on... (bootstrapping)

→ If we have an $O(n^c)$ algorithm with success probability $1-e^{-1}$, you get an algorithm with success probability $1-e^{-\lambda}$ and runtime:

$\underbrace{\lambda \frac{n(n-1)}{t(t-1)} \frac{e}{e-1}}_{\#\text{repetitions}} \cdot O(\underbrace{n(n-t)}_{\text{reduction to } t \text{ vertices}} + \underbrace{t^c}_{\text{algo over } t \text{ edges}}) = O(\lambda(\frac{n^4}{t^2} + n^{2+c-2}))$ → for $t := n^{2/c}$: Success prob. $\geq 1 - e^{-\lambda}$, runtime $O(\lambda n^{4-4/c})$

⇒ runtime goes from $n^4 \to n^3 \to n^{2.666} \to n^{2.5} \to n^{2.4} \ldots$

→ after k-time repetition, we get runtime $O(n^{2+2/k})$, which "goes to" a $O(n^2 \operatorname{polylog}(n))$ algorithm

Algorithm:

KargerStein(G) — G connected multigraph

1: if $|V(G)| \leq 6$ then
2: solve problem in constant time
3: $G' \leftarrow G$, $t := \lceil |V(G)|/\sqrt{2}+1 \rceil$
4: while $|V(G')| > t$ do
5: $e \leftarrow$ uniformly randomly selected edge in G'
6: $G' \leftarrow G'/e$
7: $C_1 :=$ KargerStein(G'), $C_2 :=$ KargerStein(G_2)
8: return the smaller set of C_1, C_2

Smallest Enclosed Disk:

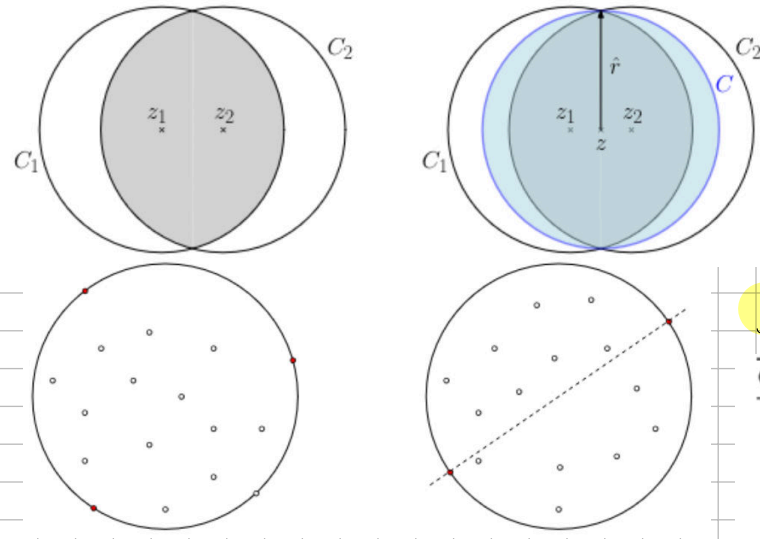
Problem: Given a finite set of points $P \subseteq \mathbb{R}^2$, compute the disk with the smallest radius that encloses P.

→ We define the closed disk bounded by a disk C as $C^\bullet$

→ C encloses P if $P \subseteq C^\bullet$, points in P can also lie on C.

Lemma: for all finite sets of points $P \subseteq \mathbb{R}^2$ there is a unique smallest enclosing disk $C(P)$.

Lemma: For any set of points $P \subseteq \mathbb{R}^2$, $|P| \geq 3$, there exists a subset $Q \subseteq P$, so that $|Q| = 3$ and $C(Q) = C(P)$ (Q is a certificate for $C(P)$)



Simple algorithms:

COMPLETEENUMERATION(P)

1: **for all** $Q \in \binom{P}{3}$ **do**
2: compute $C(Q)$
3: **if** $P \subseteq C^\bullet(Q)$ **then**
4: **return** $C(Q)$

→ We iterate over $\binom{n}{3}$ sets Q, compute $C(Q)$ in $O(1)$, and validate $P \subseteq C^\bullet(Q)$ in $O(n)$ time

⇒ overall runtime of $O(n^4)$

→ we used the fact that $\forall Q \subseteq P$, $\text{radius}(C(Q)) \leq \text{radius}(C(P)) \not\Rightarrow C^*(Q) \subseteq C^*(P)$ does not need to hold

CompleteEnumerationSmart(P)
1: $r \leftarrow 0$
2: **for all** $Q \in \binom{P}{3}$ **do** compute $C(Q)$
3: **if** radius$(C(Q)) > r$ **then**
4: $C^* \leftarrow C(Q)$; $r \leftarrow$ radius$(C(Q))$
5: **return** C^*

→ $O(n^3)$ runtime, we made use of the following facts:

→ $\forall Q \subseteq P$, $\text{radius}(C(Q)) \leq \text{radius}(C(P))$, $\forall Q \subseteq P$ with $\text{radius}(C(Q)) = \text{radius}(C(P))$: $C(Q) = C(P)$

Randomized_PrimitiveVersion(P)
1: repeat forever
2: choose $Q \subseteq P$ with $|Q| = 3$ randomly, uniformly
3: compute $C(Q)$
4: if $P \subseteq C^*(Q)$ then
5: return $C(Q)$

→ we get the correct Q with probability $\geq 1/\binom{n}{3}$

→ expected attempts until success $\leq \binom{n}{3} \Rightarrow$ expected runtime $O(n^4)$

→ Idea: we find out what points lie outside $C(Q)$ → more useful to know than the points inside of $C(Q)$

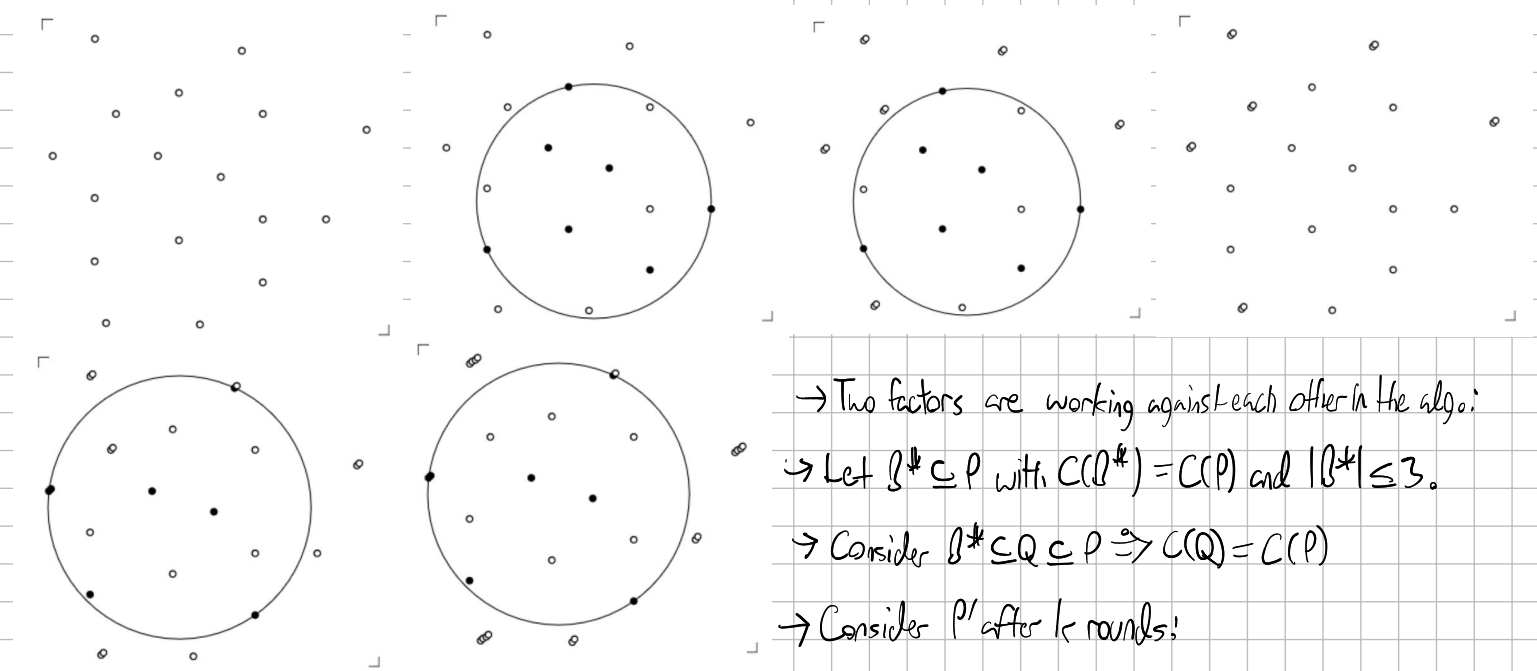
Randomized_CleverVersion(P)
1: $P' \leftarrow P$
2: repeat
3: choose $Q \subseteq P'$ with $|Q| = 11$ randomly, uniformly
4: compute $C(Q)$
5: if $P \subseteq C^*(Q)$ then return $C(Q)$
6: else double all points of P' outside of $C(Q)$

→ We require that the choice of Q is possible in $O(n)$, $n := |P|$

→ we can implement each round in $O(n)$ this way

→ how many rounds until $P \subseteq C^*(Q)$?

Algorithm in action:



→ Two factors are working against each other in the alg.:

→ Let $B^* \subseteq P$ with $C(B^*) = C(P)$ and $|B^*| \leq 3$.

→ Consider $B^* \subseteq Q \subseteq P \Rightarrow C(Q) = C(P)$

→ Consider P' after k rounds:

→ Case 1: $|P'|$ grows fast: If $P \not\subseteq C^*(Q)$, there exists a point in B^* outside of $C(Q)$ → some point in B^* must have been doubled $k/3$ times and have multiplicity $\geq 2^{k/3} \Rightarrow |P'| \geq 2^{k/3}$

→ Case 2: $|P'|$ does not grow fast: The expected value of $|P'|$ only increases by a factor $\frac{5}{4}$ (to be shown) → $\mathbb{E}[|P'|] \leq (\frac{5}{4})^k n$

Sampling Lemma: Let $r, N \in \mathbb{N}$, $r \leq N$, $P' \subseteq \mathbb{R}^2$ a multiset, $|P'| = N$. For uniformly random R from $\binom{P'}{r}$:

→ set where elements can appear multiple times

$$\mathbb{E}[\underbrace{|P' \setminus C(R)|}_{\text{points in } P' \text{ outside of } C(R)}] \leq 3\frac{N-r}{r+1} \leq 3\frac{N}{r+1}$$

Proof: for $s \in P'$, $R, Q \subseteq P'$, define indicators $\text{out}(s,R) := \begin{cases} 1, & s \notin C^*(R) \\ 0, & \text{else} \end{cases}$, $\text{ess}(s,Q) := \begin{cases} 1, & C(Q \setminus \{s\}) \neq C(Q) \\ 0 & \text{else} \end{cases}$

→ $\sum_{s \in P' \setminus R} \text{out}(s,R) = |P' \setminus C^*(R)|$, $\sum_{s \in Q} \text{ess}(s,Q) \leq 3 \Rightarrow \text{out}(s,R) = 1 \Leftrightarrow \text{ess}(s, R \cup \{s\}) = 1$

→ $\mathbb{E}[|P' \setminus C^*(R)|] = \mathbb{E}[\sum_{s \in P' \setminus R} \text{out}(s,R)] = \frac{1}{\binom{N}{r}} \sum_{R \in \binom{P'}{r}} \sum_{s \in P' \setminus R} \text{out}(s,R) = \frac{1}{\binom{N}{r}} \sum_{R \in \binom{P'}{r}} \sum_{s \in P' \setminus R} \text{css}(s, R \cup \{s\})$

$= \frac{1}{\binom{N}{r}} \sum_{Q \in \binom{P'}{r+1}} \underbrace{\sum_{s \in Q} \text{css}(s,Q)}_{\leq 3} \leq \frac{1}{\binom{N}{r}} \sum_{Q \in \binom{P'}{r+1}} 3 = 3 \cdot \frac{\binom{N}{r+1}}{\binom{N}{r}} = 3\frac{N-r}{r+1}$ ▨

→ how many rounds in the algorithm until $P \subseteq C^*(Q)$?

Amount of points after k rounds:

→ **Sampling Lemma:** $\mathbb{E}[|P' \setminus C^*(R)|] \leq 3\frac{N}{r+1}$ with $N = |P'|$

→ double the points in $P' \setminus C^*(R) \Rightarrow$ new amount of points in expected value $\leq N + 3\frac{N}{r+1} \leq (1+\frac{3}{r+1})N = \frac{5}{4}N$ for $r = 11$

→ after 0 iterations: $|P'| = n$, after one iteration: $\mathbb{E}[|P'|] \leq \frac{5}{4}n \Rightarrow$ k iterations: $\mathbb{E}[|P'|] \leq (\frac{5}{4})^k n$

Let $T \in \mathbb{N} \cup \{\infty\}$ be the **amount of rounds** of the algorithm and $X_k := |P' \setminus C^*(R)|$ for P' after $\min\{T, k\}$ rounds ($X_0 = |P| = n$).

→ $\mathbb{E}[X_k] = \sum_{t=0}^{\infty} \mathbb{E}[X_k \mid X_{k-1} = t] \cdot \Pr[X_{k-1} = t] \leq \sum_{t=0}^{\infty} (1 + \frac{3}{r+1}) t \cdot \Pr[X_{k-1} = t]$

$= (1+\frac{3}{r+1}) \cdot \sum_{t=0}^{\infty} t \cdot \Pr[X_{k-1} = t] = (1+\frac{3}{r+1}) \cdot \mathbb{E}[X_{k-1}]$

→ because $X_0 = n$, $\mathbb{E}[X_k] \leq (1+\frac{3}{r+1})^k \cdot n$

Summarized: Recall that if $T \geq k$, $X_k \geq 2^{k/3}$. We know $\mathbb{E}[X_k] \leq (1+\frac{3}{11+1})^k \cdot n = (\frac{5}{4})^k \cdot n$.

Also: $\mathbb{E}[X_k] = \underbrace{\mathbb{E}[X_k \mid T \geq k]}_{\geq 2^{k/3}} \cdot \Pr[T \geq k] + \underbrace{\mathbb{E}[X_k \mid T < k]}_{\geq 0} \cdot \Pr[T < k] \geq 2^{k/3} \cdot \Pr[T \geq k]$

$\Rightarrow 2^{k/3} \cdot \Pr[T \geq k] \leq \mathbb{E}[X_k] \leq (\frac{5}{4})^k \cdot n \Rightarrow \Pr[T \geq k] \leq \underbrace{(\frac{5}{4\sqrt[3]{2}})^k}_{\leq 0{,}993} \cdot n \leq 0.993^k n \leq 1$ (if $k \geq -\log_{0.993} n$)

Theorem: The algorithm computes the smallest enclosing disk in expected time $O(n \log n)$.

Proof: Runtime $= O(nT)$. Set $k_0 := \lfloor \log_{0.993} n \rfloor$.

$\mathbb{E}[T] = \sum_{k \geq 1} \Pr[T \geq k] \leq \sum_{k=1}^{k_0} 1 + \sum_{k < k_0} 0.993^k n = \sum_{k=1}^{k_0} 1 + \sum_{k'=0} 0.993^{k'} \cdot \underbrace{0.993^{k_0+1}}_{\leq 1}$

$= k_0 + O(1) = O(\log n)$

Convex Hull problem: Given a finite set of points $P \subseteq \mathbb{R}^2$, compute the convex hull of P.

→ Let $d \in \mathbb{N}_0$

Def.: For $v_0, v_1 \in \mathbb{R}^d$, let $\overline{v_0 v_1} := \{(1-\lambda)v_0 + \lambda v_1 \mid \lambda \in \mathbb{R}, 0 \leq \lambda \leq 1\}$ be the line segment connecting v_0 and v_1.

Def.: A set $C \subseteq \mathbb{R}^d$ is convex if $\forall v_0, v_1 \in C: \overline{v_0 v_1} \subseteq C$.

Def.: The convex hull $\text{conv}(S)$ of a set $S \subseteq \mathbb{R}^d$ is the cut of all convex sets containing S, meaning $\text{conv}(S) := \bigcap_{S \subseteq C \subseteq \mathbb{R}^d,\ C \text{ convex}} C$

→ equivalently: i) cut of all half-planes containing S, ii) smallest convex set containing S

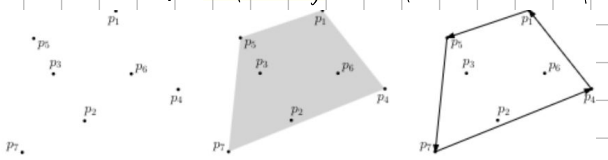
→ For a finite set of points P in a plane, the convex hull is determined by a polygon, the edge of $\text{conv}(P)$, of which the corners are points in P. Determining $\text{conv}(P)$ means computing the sequence $(q_0, q_1, \dots, q_{h-1})$, $h \leq n$ of the polygon's corners, beginning with an arbitrary edge q_0, after which the corners are computed counterclockwise along the polygon.

→ Observe that $Q_i = \{q_0, q_1, \dots, q_{h-1}\}$ is the smallest subset of P with $\text{conv}(Q) = \text{conv}(P)$.

38

Redefine the Convex Hull Problem: Given a finite set of points $P \subseteq \mathbb{R}^2$, compute the corners of the polygon surrounding conv(P) in counterclockwise order.

→ **Simplifying assumption: general position:** no 3 points on a shared straight line, no 2 points have the same x coordinate



Set of points P convex hull conv(P) Polygon(p_4, p_1, p_5, p_7)

Def.: The tuple $qr \in P^2$, $q \neq r$, is an **outer edge** of P if all points in $P \setminus \{q, r\}$ lie left of q and r, meaning they are on the left side of the directed straight line formed by q and r, directed from q to r.

Lemma: $(q_0, q_1, \dots, q_{h-1})$ is the counterclockwise **corner sequence** of the polygon surrounding conv(P) if and only if all pairs (q_{i-1}, q_i), $i \in \{1, \dots, h\}$ are outer edges of P (indices mod h)

→ Deciding if p lies left of q, r and so on → **Orientation test**

→ The area of the parallelogram spanned by $o=(0,0), p, r$ is (signed): $\|r\| \cdot \|p\| \cdot \sin\angle(r,o,p) = \det(r,p) = \begin{vmatrix} r_x & p_x \\ r_y & p_y \end{vmatrix}$

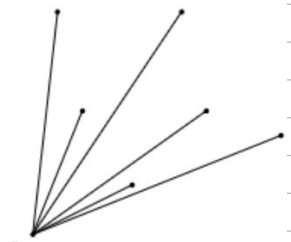
$= r_x p_y - p_x r_y$, $\sin\alpha \begin{cases} >0 \text{ if } 0^\circ < \alpha < 180^\circ \\ =0 \text{ if } \alpha = 0^\circ, \alpha = 180^\circ \\ <0 \text{ if } 180^\circ < \alpha < 360^\circ \end{cases}$ ⇒ p left of $(o,r) \Leftrightarrow r_x p_y - p_x r_y > 0$

p left of $(q,r) \Leftrightarrow p-q$ left of $(o, r-q) \Leftrightarrow (r_x - q_x)(p_y - q_y) - (p_x - q_x)(r_y - q_y) > 0$

Naive approach: Iterate over $n(n-1)$ ordered pairs qr, check if it is an outer edge by testing if p lies left of qr for all $n-2$ points p in $P \setminus \{q, r\}$ ⇒ we find outer edges in $O(n^3)$, just need to order them

Finding the next point: q_0 := point with smallest x coordinate in P ⇒ corner of convex hull, begin sequence with q_0 → how to find q_1?

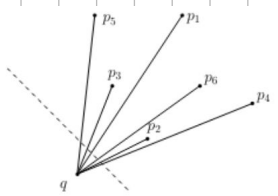
Algorithm: FindNext(q)
1: Randomly choose $p_0 \in P \setminus \{q\}$
2: $q_{next} \leftarrow p_0$
3: for all $p \in P \setminus \{q, p_0\}$ do
4: if p right of q, q_{next} then
5: $q_{next} \leftarrow p$
6: return q_{next}



Order around a point:

→ Given $q \in P$, let $\prec_q$ be a relation over $P \setminus \{q\}$ with:

$p_1 \prec_q p_2 :\Leftrightarrow p_1$ right of q and p_2



$p_4 \prec_q p_2 \prec_q p_6 \prec_q p_1$
$\prec_q p_3 \prec_q p_5$

qp_4 is outer edge.

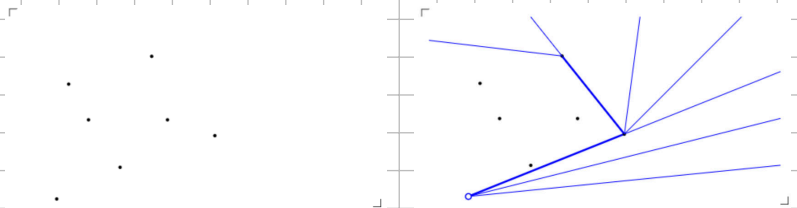
Lemma: If q is a corner of the convex hull of P, the relation $\prec_q$ is a **total order** over $P \setminus \{q\}$. For the minimum p_{min} of this order, (q, p_{min}) is an outer edge.

Algorithm: JarvisWrap(P) (I'm sorry, this is hilarious to me)

Jarvis wrap.
1: $q_0 \leftarrow$ point in P with smallest x coordinate
2: $h \leftarrow 0$
3: repeat
4: $h \leftarrow h+1$
5: $q_h \leftarrow$ FindNext(q_{h-1})
6: until $q_h = q_0$
7: return $(q_0, q_1, \dots, q_{h-1})$

Theorem: For a set P of n points in $\mathbb{R}^2$ in general position, the algorithm JarvisWrap computes the convex hull in time $O(nh)$, with h being the amount of corners of the convex hull P.

Visual Example:



→ as $h \leq n$, JarvisWrap runs in $O(n^2)$.

→ if $h = O(1)$, like for triangle-shaped conv(P), it runs in $O(n)$, for random points in square: expected # corners $O(\log n)$, for circles: $O(\sqrt[3]{n})$

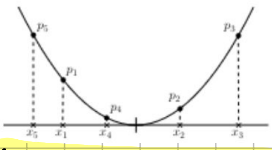
Caveats: Collinearity (3 points on straight line) / same x coordinate / starting point q_0 has lexicographically smallest coordinate

(smallest y coordinate among points with lowest x coordinate), test "p right of q and q_{next}" needs to be replaced by "p right of q and q_{next} or p on straight line through q and q_{next} and $|qp| > |qq_{next}|$"

→ all of these need to be considered!

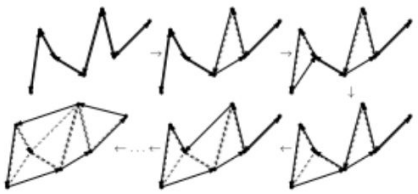
→ numeric issue: "$(r_x - q_x)(p_y - q_y) - (p_x - q_x)(r_y - q_y) > 0$" not exact for floats → algo. can return completely wrong results or get stuck, example: ignoring starting point because it appears twice → program libraries give exact data types

Lower bound for ConvexHull: Consider a sequence $\langle x_1, \dots, x_n \rangle$ in $\mathbb{R}$. Set $p_i = (x_i, x_i^2)$, $i \in \{1, \dots, n\}$ (vertical projection of x axis in $\mathbb{R}^2$ onto unit parable $y = x^2$)

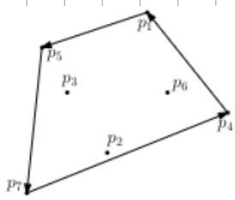


→ From the sequences of corners of $P := \{p_1, p_2, \dots, p_n\}$ the increasing sorted order of x_i's can be computed in linear time → Reduction: if we can solve ConvexHull in $t(n)$, we can sort in $t(n) + O(n)$ time.

Locally improving the Convex Hull:

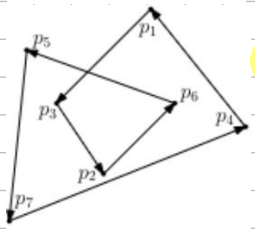


→ Recall the definition of an outer edge → Local test in polygon $\langle q_0, q_1, \dots, q_{h-1} \rangle$:

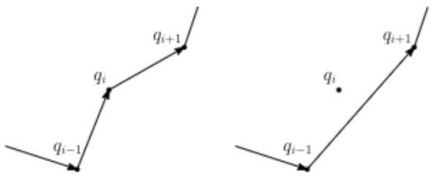


→ $\forall i,\ i \in \{1, \dots, h\}$: q_{i+1} left of q_{i-1} and q_i

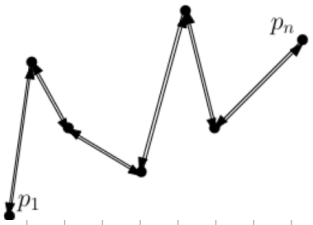
→ $\{q_0, \dots, q_{h-1}\}$ doesn't have to be a subset of P, the polygon must not have all points inside of it, it may also cross itself → Idea: start with any polygon fulfilling these conditions, then convexify it (local improvement)

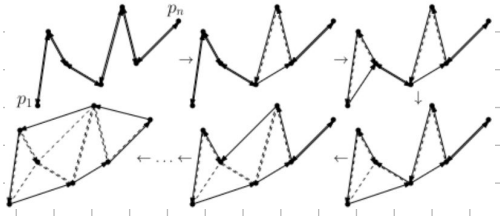


Local improvement: Given $\langle q_0, \dots, q_{k+1} \rangle$, remove q_i from the sequence if q_{i+1} right of q_{i-1} and q_i



Starting polygon: Sort P in increasing order by x coordinate $\langle p_1, p_2, \dots, p_n \rangle$ and consider the polygon $\langle p_1, p_2, \dots, p_{n-1}, p_n, p_{n-1}, \dots, p_2 \rangle$





Invariants during local improvements:

→ The partial polygon $\langle p_1, \dots, p_n \rangle$ is x-monotonous (from left to right) and contains no point in P.

→ The partial polygon $\langle p_n, \dots, p_1 \rangle$ is x-monotonous (from right to left) and contains no point in P.

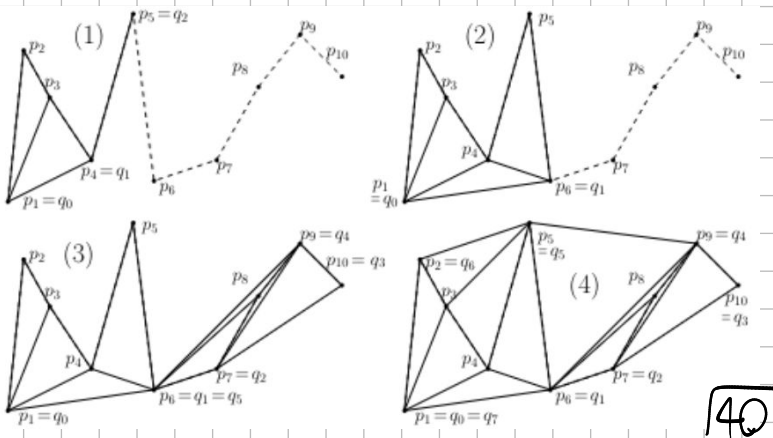
→ The partial polygon $\langle p_1, \dots, p_n \rangle$ is not above the partial polygon $\langle p_n, \dots, p_1 \rangle$ anywhere.

⇒ The polygon cannot cross itself and encloses all points ⇒ locally convex ⇒ solution polygon

→ non-deterministic algorithm → local improvement steps may be taken in an arbitrary order

Algorithm: LocalRepair$(p_1, \dots, p_n)$, $\langle p_1, \dots, p_n \rangle$ sorted

```
1: q_0 ← p_1, h ← 0
2: for i ← 2 to n do                ▷ lower edge, left to right
3:     while h > 0 and p_i right of q_{h-1} and q_h do
4:         h ← h − 1
5:     h ← h + 1; q_h ← p_i
6:     ▷ ⟨q_0, ..., q_h⟩ lower convex hull of {p_1, ..., p_i}
7: h' ← h
8: for i ← n − 1 down to 1 do       ▷ upper edge, right to left
9:     while h > h' and p_i right of q_{h-1} and q_h do
10:        h ← h − 1
```



40

11: $h \leftarrow h+1$; $q_h \leftarrow p_i$;
12: return $(q_0, \dots, q_{h-1})$

Analysis:

→ we start with a polygon with $2(n-1)$ edges and have h edges in the end → improve $2(n-1)-h = O(n)$ times

→ there are $O(n)$ successful tests "p_i right of q_{h-1} and q_h" and 2 unsuccessful tests (one below, one above) for each point p_i.

⇒ Algorithm has runtime $O(n)$ after sorting in $O(n \log n)$

Theorem: Given a sequence $p_1, \dots, p_n$ of points in $\mathbb{R}^2$ sorted by x coordinate in general position, the algorithm LocalRepair computes the convex hull of $\{p_1, \dots, p_n\}$ in time $O(n)$.

→ Sorting is a lower bound for the convex hull problem → LocalRepair is optimal

This concludes the theoretical portion of the recap, we finished the script as of May 26!

Exam Tasks for Practice: (Theory highlights from FS21):

Task 3: Fishes in a Pond: (Coupon Collector Routine)

You have a pond with n salmon and a trout. You use a net to catch fish from the pond. Whenever you catch a fish, it is caught uniformly and randomly from the fish still in the pond. For a grill party, you want to catch all salmon. Because you want to maintain the population of trout, you release any trout you catch. Prove that you expect to catch $\sum_{i=1}^{n} \frac{n+i}{i}$ fish to catch all salmon.

Solution: Define the random variable X as the amount of attempts to catch all n salmon. We are looking for $\mathbb{E}[X]$. We separate the analysis into n phases and catch one salmon per round (parallels to Coupon Collector?), so we define the n random variables X_i for $0 \le i \le n-1$ as the amount of attempts for phase i → to catch the $i+1$-th salmon. Therefore, $X = \sum_{i=0}^{n-1} X_i$ and $\mathbb{E}[X] = \mathbb{E}[\sum_{i=0}^{n-1} X_i] = \sum_{i=0}^{n-1} \mathbb{E}[X_i]$.
Because we have caught i salmon, there are still $n-i$ salmon in the pond and because we always release trout, there are n trout in the pond ⇒ we catch a salmon with probability $p_i = \frac{n-i}{2n-i}$ per attempt.
Because we are trying for the first success, we have a geometric distribution with probability p_i
⇒ $\mathbb{E}[X_i] = \frac{1}{p_i} = \frac{2n-i}{n-i}$ ⇒ $\mathbb{E}[X] = \sum_{i=0}^{n-1} \frac{2n-i}{n-i} = \sum_{j=1}^{n} \frac{n+j}{j}$ (set $j = n-i$) ∎

Matchings:

a) fill the gaps in the following algorithm s.t. it finds a maximal matching in $G=(A \dot\cup B, E)$ with $|A|=|B|=n$

```
function Maximal_Matching(G):   (this is just Hopcroft-Karp!)
  M := {e} for some edge e ∈ E
  while there is an augmenting path do:
    k := length of shortest augmenting path
    Find a maximal set S of
      pairwise disjoint augmenting paths of length k
    for all P aus S do
      M = M ⊕ P
  return M
```

b) in big O notation, give an upper bound of how often the while loop is executed and explain your answer:

Solution: Upper bound → $O(\sqrt{n})$ → Proof: after every run of the loop, the length k of the shortest path increases by 2 → especially, after $\sqrt{n}$ runs, $k \ge 2\sqrt{n}$. Also, importantly, for any matching M', $|M'| \le |M| + |V|/(k+1)$.
⇒ if $k \ge \sqrt{n}$, the maximal matching has at most $\sqrt{n}$ more edges than M_0. Because the matching increases in size by 1 after every iteration, we only need $\sqrt{n}$ iterations until the algorithm terminates ⇒ $\sqrt{n} + \sqrt{n} = O(\sqrt{n})$ iterations.

c) Let M be the algorithm's output. Let $A' \subseteq A$ and $B' \subseteq B$ be the set of edges of G that are not covered by M. Show that $|A'| = |B'|$ and that for every pair $a \in A'$ and $b \in B'$

Solution: Every matching M covers exactly $|M|$ edges from A and B ⇒ Because $|A|=|B|$, $|A'| = |A| - |M| = |B| - |M| = |B'|$.
Because the algorithm returns a maximal matching, A' and B' share no edges. Assume there is $a \in A', b \in B'$ with $\deg(a) + \deg(b) > |A| + |A'|$. We will show that there must be an augmenting path of length 3 from a to b,

Contradicting the maximality of M and concluding the proof. We know that $N(a) \subseteq B \setminus B'$, analogously $N(b) \subseteq A \setminus A'$, meaning that every outgoing edge of a and b is incident to an edge of M. $\Rightarrow$ More than $|A|-|A'| = |M|$ edges are leaving a and b. By the pigeonhole principle, there is an edge $e = \{u,v\} \in M$ that is incident to 2 edges of a or b. Because the graph is not a multigraph, one of them is outgoing from a and the other from b $\not\downarrow$ w.l.o.g., these edges are $\{a,v\}, \{w,b\}$ $\rightarrow$ augmenting path of length 3 ■

Smallest Enclosing Disk:

Let P be a set with n points on the plane and let $(p_1, \dots, p_n)$ be a uniformly randomized sequence of P.

a) give the best possible upper bound as a function of n giving the probability that p_n is outside of $C(\{p_1, \dots, p_{n-1}\})$.

Solution: for p_n to be outside $C(P \setminus \{p_n\})$, $C(P \setminus \{p_n\}) \neq C(P)$ must hold $\Rightarrow$ p_n ust be essential with respect to P. As there are 3 such points in any finite-point set P, and p_n is chosen uniformly, randomly from $P \rightarrow$ probability at most $3/n$ ■

Consider the following auxiliary problem: Let P, H be 2 finite sets of points with $|H| = 0, 1, 2$ or 3. Define $C(P,H)$ as the smallest enclosing disk of P with H on its edge. Such a disk must not always exist, and if it does, it is not automatically identical to the smallest enclosing disk of $P \cup H$.

b) Assume that there exists an algorithm that computes $C(Q \cup \{x\})$ for any valid combination of a set of points Q and a point x in expected runtime $O(|Q|)$. Describe how you can compute $C(\{p_1, \dots, p_n\})$ using $C(\{p_1, \dots, p_{n-1}\})$. Describe an algorithm taking $\{p_1, \dots, p_n\}$ as an input that returns $C(\{p_1, \dots, p_n\})$ in $O(n)$ time. (and prove it)

Solution: In the case that $C(\{p_1, \dots, p_{n-1}\})$ is known, we check if p_n is already enclosed by the circle, in which case we return it. Else, we call the helper algorithm to compute $C(\{p_1, \dots, p_{n-1}\}, p_n)$ and return it.

If p_n is enclosed by $C := C(\{p_1, \dots, p_{n-1}\})$ it is trivially an enclosing circle of P. Because adding points cannot make the circle smaller, $C = C(P)$. Else, if p_n is outside of C, p_n is on the boundary of $C(P) \Rightarrow C(P \setminus \{p_n\}, p_n)$ exists and is equal to $C(P)$. All of the earlier computations above except for computing $C(P \setminus \{p_n\}, p_n)$ can be done in $O(1)$ time. As the vertices are ordered uniformly at random, we know (from a)) that the probability of having to call the helper algorithm to compute $C(P \setminus \{p_n\}, p_n)$ is at most $3/n$. $\Rightarrow$ Takes $O(n) \cdot 3/n$ time in expectation $\Rightarrow O(1)$ time. We can easily extend this idea to an algorithm that computes $C(P)$ in expected time $O(n)$: Let C_i be the smallest enclosing disk of the first (w.r.t. our random order) i points in P. Initially, compute C_3 by case distinction. Then, iteratively use the procedure above to extend C_i to C_{i+1} for $i = 3, 4, \dots, n-1$. Correctness follows from the correctness of the procedure above. As the ordering of vertices in P is uniform and random, any internal order of the first i vertices is equally likely $\rightarrow O(1)$ expected runtime applies for each extension step, giving a total runtime of $O(n)$. ■

c) To finish the proof, we want to use the algorithm from b). Describe a randomized algorithm computing $C(Q, \{x\})$ with probability of success 1 and expected runtime $O(|Q|)$.

You can use the following geometric facts without proof (Hint):

1) for 3 points $p_1, p_2, p_3 \in P$ there is a smallest enclosing disk with $\{p_1, p_2, p_3\}$ on the edge.

2) If for a set of points P, a (not necessarily smallest) enclosing disk C with H on the edge exists, $C(P, H)$ exists and is unique.

3) For sets of points P and H, there are at most 3 points $p \in P$ s.t. $C(P \setminus \{p\}, H) \neq C(P, H)$. All of these points are on the edge of $C(P, H)$.

Solution: The idea is to repeat the algorithm from b). If $\exists$ algorithm to compute $C(Q, \{x, y\})$ in (expected) linear time, we can use it like in b) to compute $C(Q, \{x\})$. Analogously, if $\exists$ an algorithm to compute $C(Q, \{x, y, z\})$ in expected linear time, we can use it to compute $C(Q, \{x, y\})$. $C(Q, \{x, y, z\})$ is simple as there is only one circle with x, y, z on its boundary. Let P and H be any finite sets of points s.t. $|H| \leq 3$ and $C(P, H)$ exists.

```
function GetC(P, H)
    if |H| = 3 or |P| = 0 then
        Compute C(P, H) directly and return.
    p ← random point from P chosen uniformly
    C ← GetC(P \ {p}, H)
    if p enclosed by C then
        return C
    else return GetC(P \ {p}, H ∪ {p})
```

We show by induction on $|P|$ that this gives the correct output.

Trivial for $P = \emptyset$ or $|H| = 3$. Assume $|P| = i+1$, $|H| \leq 2$, and that the algorithm is correct for any valid set of points of size i.

As (P, H) is a valid input, by the 2nd hint, $C(P \setminus \{p\}, H)$ exists for any $p \in P$. If the circle encloses p, we are done. Else, by the 3rd hint, p is on the boundary of $C(P, H)$, so this circle is also the smallest enclosing disk of the points $P \setminus \{p\}$ with $H \cup \{p\}$ on its boundary. Finally, let $T_{n,k}$ denote the max. runtime of the algorithm for any valid pairs of sets P, H, where $|P| = n$ and $|H| = k$. By construction, $T_{n,3} = O(1)$. Furthermore, for $k \leq 2$: $T_{n,k} \leq T_{n-1,k} + O(1) + T_{n-1,k+1} \cdot \frac{3}{n}$, where $3/n$ follows from the 3rd hint. By substituting $k = 2, 1, 0$ iteratively, $T_{n,k} = O(n)$. ∎

43